=5textblock*env/textblock*/after

REPORTE DE PRÁCTICA NO. 1

Análisis de Expresiones Regulares

ÁREA ACADÉMICA:
Área Académica de Computación y Electrónica

Licenciatura en Ciencias Computacionales

Bucio Hidalgo Pilar

CATEDRÁTICO:
Eduardo Cornejo Velázquez

1. Introducción

Una **Gramática Formal** definida sobre un alfabeto Σ es una tupla de la forma:

$$G = \{\Sigma_T, \Sigma_N, S, P\}$$

donde:

- Σ_T es un alfabeto de símbolos terminales
- Σ_N es un alfabeto de símbolos no terminales
- S es el símbolo inicial de la gramática
- P es un conjunto de producciones gramaticales

Además, se cumple:

$$S \in \Sigma_N$$

$$\Sigma_T \cap \Sigma_N = \emptyset$$

$$\Sigma = \Sigma_T \cup \Sigma_N$$

La gramática formal G permite generar un lenguaje $L = \{x \in \Sigma_T^* \mid S \to^* x\}$.
Por lo tanto, las palabras del lenguaje estarán formadas por cadenas de símbolos terminales generadas a partir del símbolo inicial de la gramática utilizando las producciones que la definen.
Una **expresión regular** es una notación normalizada para representar lenguajes regulares, es decir, lenguajes generados por gramáticas regulares (Tipo 3). Las expresiones regulares permiten describir con exactitud y sencillez cualquier lenguaje regular.

2. Marco teórico

El marco teórico de esta práctica se fundamenta en los conceptos básicos del análisis léxico y el procesamiento de expresiones regulares en el lenguaje Java.

Análisis Léxico, Tokens y Lexemas

El análisis léxico representa la primera fase en el procesamiento de un lenguaje. Consiste en leer la secuencia de caracteres de un texto de entrada para identificar unidades con significado:

- **Token:** Es la categoría abstracta o tipo de elemento identificado (por ejemplo: *Número entero*, *Identificador*, *Símbolo*).
- **Lexema:** Es la cadena de texto concreta o secuencia de caracteres que coincide con el patrón del token (por ejemplo: `2026`, `variable1`, `+`).

Procesamiento de Expresiones Regulares en Java

Para evaluar y clasificar los lexemas dentro de una cadena de texto, Java proporciona el paquete `java.util.regex`, apoyándose en dos clases principales:

- **Pattern:** Permite definir y compilar la expresión regular que especifica la regla de formación de cada categoría léxica.
- **Matcher:** Se encarga de examinar el texto de entrada para comparar la secuencia de caracteres contra la expresión regular y determinar si existe una coincidencia exacta.

3. Obejtivo General

Construir programas en un lenguaje de programación de alto nivel que realicen el análisis de cadenas de símbolos que forman palabras basado en la implementación de estructuras de datos y algoritmos de análisis, además del uso de expresiones regulares como segunda estrategia de implementación.

Objetivos Específicos

- Identificar el proceso básico de análisis de cadenas de símbolos de un alfabeto.
- Implementar estructuras de datos y algoritmos de análisis de cadenas de símbolos en un lenguaje de programación de alto nivel.
- Implementar expresiones regulares para el análisis de cadenas de símbolos en un lenguaje de programación de alto nivel.

6.Desarrollo de la Pràctica

Punto 1: Análisis de cadenas de símbolos con estructuras de datos

Para este primer paso de la practica lo que se hizo fue escoger un lenguaje de programacion,para construir los programas basados en estructuras de datos para analizar cadenas de simbolos,con el proposito de identificar las siguientes puntos.

- **Número entero:**
- **Palabras en minúsculas:**
- **Palabras en mayúsculas:**
- **Identificadores (Nombres de variables):**

para la creaciòn de cada uno de los programas se escogio el lenguaje de programaciòn de java.El cual nos ayudara con el analisis de cada una de las cadenas que se utilizaran para el reconocimiento de cada uno.

Punto 2: Definición del alfabeto y Expresiones Regulares

Se definió el alfabeto general de entrada como $\Sigma = \{a..z, A..Z, 0..9, _, +, -, =, *, /, ;, \ldots\}$. Para realizar se tuvo que buscar o reconocer los elementos de cada uno de los programas que se haran para ver que tipo de caracteres son vàlidos o no. Como tambien formar las expresiones regulares que se forman en base a nuestro alfabeto.
Esto nos va a servir en el momento de compilar el programa ya que, el programa va a detectar si el usuario ingreso un simbolo que no esta dentro de nuestro alfabeto y detecta que esa palabra, simbolo, etc. No es valida

Categoría	**Expresión Regular**
Números enteros	`^[0-9]+$`
Palabras en minúsculas	`^[a-z]+$`
Palabras en mayúsculas	`^[A-Z]+$`
Identificador	`^[a-zA-Z_][a-zA-Z0-9_]*`
Símbolos	`^[^\w\s]$`

Table 1: Alfabeto y expresiones regulares definidas.

Punto 3: Clasificación y contabilización mediante expresiones regulares

Para esta fase se creo el programa en Java utilizando las clases `Pattern` y `Matcher` del paquete `java.util.regex`. El algoritmo realiza el procesamiento en las siguientes etapas:

1. **Tokenización:** Se escanea la cadena completa introducida por el usuario.
2. **Evaluación y Clasificación:** Cada token se evalúa contra las expresiones regulares. En cuanto coincide con alguna regla, se asigna su tipo correspondiente dependiendo lo que sea.
3. **Contabilización:** Se utiliza una serie de contadores para contar y ver cuantos elementos de cada categoria hay.
4. **Despliegue de datos:** Se presentan los resultados al usuario mediante la compilacion del programa.

Cuestionario

1. **¿Cuál es la utilidad de las expresiones regulares en la identificación de palabras de un lenguaje?**

   Permiten definir patrones, es decir son los que nos ayudaran a que el analizador léxico se le facilite buscar y validar los caracteres que se estan metiendo

2. **¿Cuáles elementos del lenguaje de programación utilizaste para la implementación de las estructuras de datos?, ¿Cómo las utilizaste?**

   Se utilizaron cadenas de texto (`String`) y arreglos de cadenas (`String[]`). En los programas ejecutados por consola, se empleó la estructura de datos tipo arreglo para fragmentar y almacenar las palabras individuales obtenidas de la entrada, permitiendo recorrerlas mediante un ciclo.

3. **¿Cuál es el proceso que seguiste para analizar las cadenas de símbolos?**

4. **1.Lectura de entrada:** Se obtiene la cadena de caracteres introducida por el usuario a través de la consola

5. **Tokenización:** Se divide el texto en elementos individuales

6. **Evaluación secuencial:** Se recorre cada token mediante un ciclo condicional y se compara contra las expresiones regulares

7. **Clasificación y despliegue:** Se determina la categoría coincidente (Número entero, Minúscula, Mayúscula, Identificador o Símbolo), imprimiendo la clasificación de cada palabra en la terminal y contando cuantas de estas hay palabra en la terminal y contando cuantas de estas hay.

¿Qué elementos del lenguaje de programación te permitieron implementar expresiones regulares?, ¿qué condiciones o recomendaciones se deben seguir al escribir las expresiones regulares?
Se utilizó el paquete nativo `java.util.regex` y creo que al escribir expresiones regulares, es recomendable saber el inicio y el fin del token.
Y el hecho de la simplicidad que lleve la expresión, entre más corta y directa, mucho mejor.

Resultados

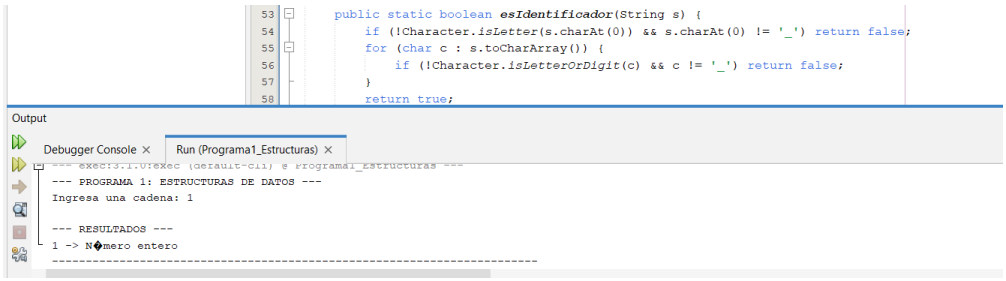


Figure 1: Ejecución y resultados del Programa 1 en consola MAYUSCULAS.

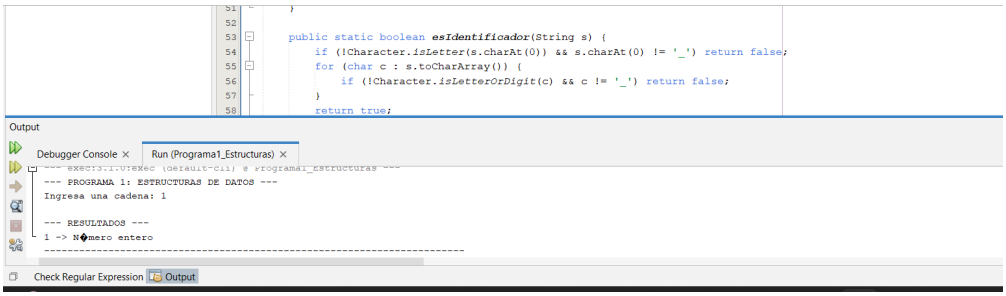


Figure 2: Ejecución y resultados del Programa 1 en consola ENTEROS

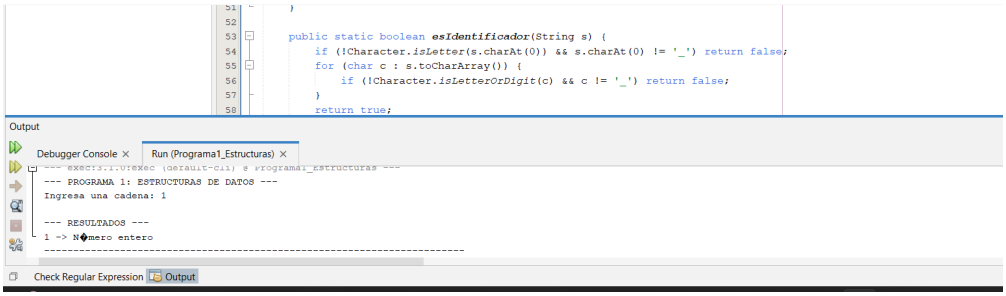


Figure 3: Ejecución y resultados del Programa 1 en consola SIMBOLOSx

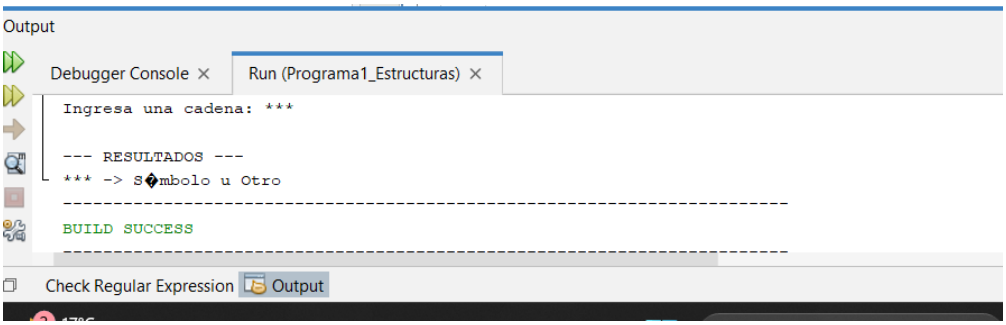


Figure 4: Ejecución y resultados del Programa 1 en consola SIMBOLOS

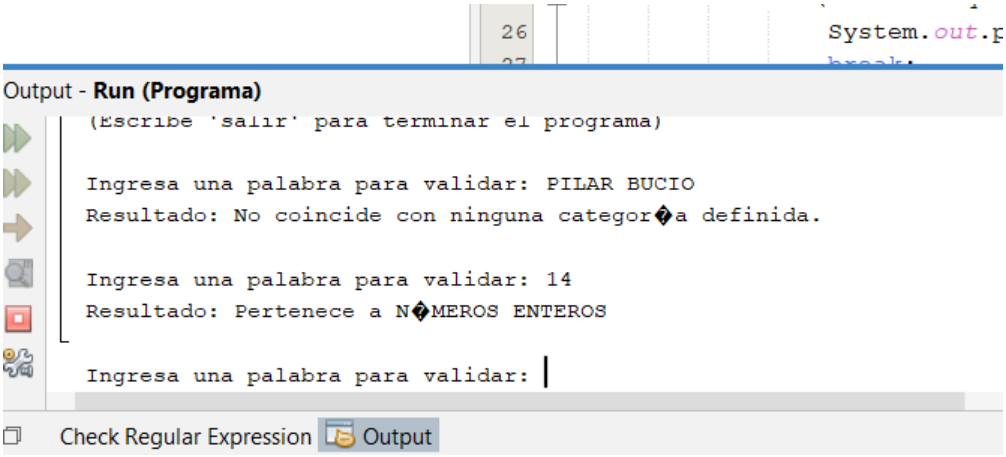


Figure 5: Ejecución y resultados del Programa 2 en consola

Conclusiones

El desarrollo de esta práctica me ayudó a entender de forma mucho más clara cómo funcionan las expresiones regulares y la importancia del análisis léxico en la programación. Al principio fue algo confuso lidiar con las reglas de validación y la sintaxis de las regex, pero al ir probando cada caso con diferentes datos (números, palabras, identificadores y símbolos) pude ver exactamente cómo Java va reconociendo y clasificando cada tipo de token.

Bibliografia

References

[1] Eduardo Cornejo Velasquez *Manual de Prácticas de Lenguajes y Autómatas*, Instituto de Ciencias Basicas y Ingieneria, 2026.